



# **HSB**

Hochschule Bremen  
City University of Applied Sciences

## **REISESYSTEM DOKUMENTATION**

**INTERNATIONALER FRAUENSTUDIENGANG INFORMATIK**

**MODUL: DATENBANKEN**

**PROF. DR. UTA BOHNEBECK**

**WISE 2024/2025**

**Autorinnen:**

**Sarah Zahed (5315435)**

**Aylin Boynukara (5318634)**

**Jana Brüggemann (5310302)**

# Inhaltsverzeichnis

|                                                                                                                                                                                                         |           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>Einleitung.....</b>                                                                                                                                                                                  | <b>2</b>  |
| <b>Das Entity Modell.....</b>                                                                                                                                                                           | <b>2</b>  |
| Unser Modell.....                                                                                                                                                                                       | 2         |
| Die Erläuterung zum Modell.....                                                                                                                                                                         | 3         |
| Normalformen.....                                                                                                                                                                                       | 4         |
| <b>Das Relationale Modell.....</b>                                                                                                                                                                      | <b>12</b> |
| Beschreibung.....                                                                                                                                                                                       | 16        |
| <b>Unser Skript.....</b>                                                                                                                                                                                | <b>18</b> |
| Trigger 1.....                                                                                                                                                                                          | 18        |
| Trigger 2.....                                                                                                                                                                                          | 18        |
| Trigger 3.....                                                                                                                                                                                          | 19        |
| Trigger 4.....                                                                                                                                                                                          | 19        |
| Trigger 5.....                                                                                                                                                                                          | 19        |
| Trigger 6.....                                                                                                                                                                                          | 20        |
| <b>Testszenarios mit SQL.....</b>                                                                                                                                                                       | <b>21</b> |
| Einfache Select-Statements.....                                                                                                                                                                         | 21        |
| Updates.....                                                                                                                                                                                            | 29        |
| Joins.....                                                                                                                                                                                              | 29        |
| Löschen.....                                                                                                                                                                                            | 33        |
| Aggregat.....                                                                                                                                                                                           | 35        |
| Group-by-Klausel.....                                                                                                                                                                                   | 37        |
| Geschachteltes Select-Statement.....                                                                                                                                                                    | 38        |
| Erfolgreiche Transaktion.....                                                                                                                                                                           | 38        |
| Abgebrochene Transaktion.....                                                                                                                                                                           | 39        |
| <b>Beantwortung der theoretischen Fragen.....</b>                                                                                                                                                       | <b>41</b> |
| 1. Was versteht man im Datenbankkontext unter einer Transaktion?.....                                                                                                                                   | 42        |
| 2. Erläutern Sie das ACID-Paradigma.....                                                                                                                                                                | 42        |
| 3. Welche (drei) Concurrency-Probleme können bei einem verteilten Datenbankbetrieb auftreten? Erläutern Sie diese (inkl. ihres zeitlichen Verlaufs) jeweils anhand eines selbstgewählten Beispiels..... | 42        |
| 4. Was versteht man unter einem S-Lock, einem X-Lock und einem Deadlock?.....                                                                                                                           | 44        |
| 5. Was versteht man im Datenbankkontext unter dem Begriff „Zugriffslücke“?.....                                                                                                                         | 44        |

# Einleitung

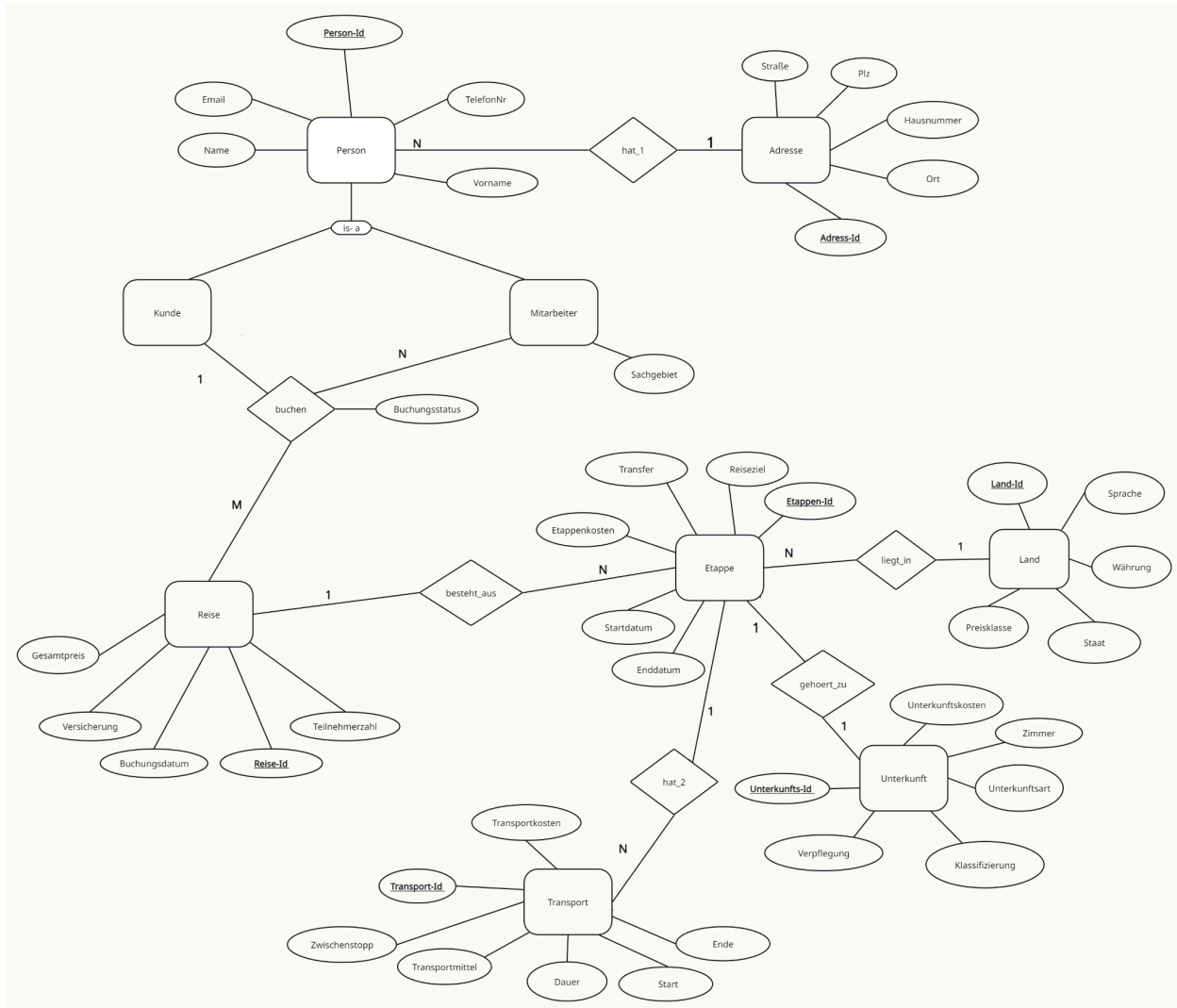
Unsere Datenbank umfasst die Entwicklung und Umsetzung einer relationalen Datenbank für ein Reisesystem. Ziel ist es, eine strukturierte und effiziente Speicherung sowie Verarbeitung von Daten rund um Reisen, Kunden, Mitarbeiter und Buchungen zu ermöglichen. Die Datenbank bildet verschiedene Entitäten wie Personen, Reisen, Etappen, Unterkünfte und Transportmittel ab und stellt die Beziehungen zwischen diesen dar. Dabei wird besonders auf die Normalisierung der Daten geachtet, um Redundanzen zu vermeiden und eine konsistente Datenstruktur zu gewährleisten. Ein wesentlicher Bestandteil dieses Projekts ist die Automatisierung wiederkehrender Prozesse mithilfe von Triggern und Funktionen. Zusätzlich werden Transaktionen eingesetzt, um sicherzustellen, dass fehlerhafte Eingaben rückgängig gemacht werden können.

In dieser Dokumentation werden die entstandenen Tabellen, Beziehungen und Funktionen beschrieben. Zudem werden die durchgeführten SQL-Abfragen und deren Ergebnisse analysiert.

## Das Entity Modell

### Unser Modell

Hinweis: Eine PDF-Version des ER-Modells befindet sich in der ZIP-Datei. Diese Version bietet eine höhere Qualität, da die hier eingebundene Darstellung aufgrund der Komprimierung leicht an Schärfe verlieren kann.



## Die Erläuterung zum Modell

Das Modell basiert auf verschiedenen Entity-Typen, die miteinander in Beziehung stehen. Es gibt eine Generalisierung, die Entität 'Person'. Sie hat die Attribute:

- Primärschlüssel: Person-Id
- Name, Vorname, E-Mail und Telefonnummer

Eine 'Person' besitzt genau eine 'Adresse', wobei mehrere Personen dieselbe Adresse haben können, beispielsweise in einem Wohnblock. Dies wird ersichtlich an der Beziehung 'hat\_1' zwischen der Entität 'Person' und 'Adresse' (N:1).

Adresse besitzt die Attribute:

- Primärschlüssel: Adresse-Id
- Straße, Postleitzahl, Hausnummer und Ort.

Die Entität 'Person' kann weiter spezialisiert werden. Es herrscht eine "Is a" Beziehung, wodurch eine Vererbung erkennbar wird:

- 'Kunde' erbt von 'Person' und hat keine zusätzlichen Attributen
- 'Mitarbeiter' erbt ebenso von 'Person' und hat das zusätzliche Attribut Sachgebiet

Sowohl 'Kunden' als auch 'Mitarbeiter' stehen mit der Entität 'Reise' in einer Beziehung:

- buchen (*buchen*-Beziehung): Jede Buchung enthält zusätzlich das Attribut 'Buchungsstatus'.
- Kunde bucht Reise (1:M): Ein Kunde kann mehrere Reisen buchen, aber eine Reise gehört immer genau einem Kunden.
- Mitarbeiter bucht Reise (N:M): Mitarbeiter können mehrere Reisen für Kunden buchen, da es mehrere Mitarbeiter und Kunden gibt.

Die Entität 'Reise' hat

- als Primärschlüssel die Reise-Id
- und die Attributen, Gesamtpreis, Versicherung, Buchungsdatum, Teilnehmerzahl

Sie hat eine Beziehung 'besteht\_aus' zu der Entität Etappe, die:

- als Primärschlüssel die Etappen-Id
- und die Attribute Enddatum, Startdatum, Etappenkosten, Transfer und Reiseziel hat.

Eine Reise kann aus mehreren Etappen bestehen, deshalb gilt hier die Kardinalität 1:N. Von der Entität 'Etappe' gehen drei Beziehungen aus:

- hat\_2 mit Transport (1:N): eine Etappe kann mehrere Transporte umfassen
- gehoert\_zu mit Unterkunft (1:1): Jeder Etappe ist genau eine Unterkunft zugeordnet. Beispiel: Bei einem dreitägigen Aufenthalt in Rom bleibt die Unterkunft dieselbe.
- liegt\_in mit Land (N:1): eine Etappe findet in genau einem Land statt, aber ein Land kann mehrere Etappen umfassen (z. B. erst Ankara, dann Istanbul, dann Antalya).

## Normalformen

Um später das ER-Modell in das Relationale Modell überführen zu können, muss man es zunächst in die Normalform überführen. Dabei nutzen wir die dritte Normalform von Boyce und Codd. Diese Normalform besagt, dass jede Determinante ein Schlüsselkandidat sein muss. Das bedeutet, dass es keine funktionalen Abhängigkeiten zwischen Nicht-Schlüsselattributen geben darf, die nicht durch einen Schlüsselkandidaten bestimmt werden. Bei der Überführung des ER-Modell in die dritte Normalform, wird sowohl für jede Entität als auch zu jeder Relation eine Tabelle erstellt. Dies sieht wie folgt aus:

Person

| <u>Person-Id</u> | Name         | Vorname   | Email                   | TelefonNr |
|------------------|--------------|-----------|-------------------------|-----------|
| 'K001'           | 'Müller'     | 'Klaus'   | 'muellerchen@gmail.com' | 12345     |
| 'M002'           | 'Mustermann' | 'Max'     | 'm.Mueller@gmail.com'   | 678910    |
| 'K003'           | 'Kruse'      | 'Otto'    | 'kruOtto@gmail.com'     | 111213    |
| 'M004'           | 'Lampe'      | 'Hannah'  | 'h.Lampe@gmail.com'     | 141516    |
| 'K005'           | 'Parker'     | 'Lisa'    | 'LiParker@gmail.com'    | 171819    |
| 'M006'           | 'Can'        | 'Selina'  | 'S.Can@gmail.com'       | 202123    |
| 'K007'           | 'Choi'       | 'Kai'     | 'choi.kai@gmail.com'    | 242526    |
| 'M008'           | 'Lomberg'    | 'Mareike' | 'm.Lomberg@gmail.com'   | 282930    |

Adresse

| <u>Adress-Id</u> | Straße             | Hausnummer | PLZ   | Ort        |
|------------------|--------------------|------------|-------|------------|
| 1                | 'Knochenstraße'    | '3'        | 24111 | 'Bremen'   |
| 2                | 'Sandstraße'       | '29'       | 29141 | 'Bremen'   |
| 3                | 'Palaststraße'     | '16'       | 20221 | 'Bremen'   |
| 4                | 'Schulstraße'      | '8'        | 22222 | 'Hamburg'  |
| 5                | 'Taunusstraße'     | '7'        | 41321 | 'Muenchen' |
| 6                | 'Spandauer Straße' | '5'        | 33123 | 'Berlin'   |
| 7                | 'Baumstraße'       | '106'      | 52675 | 'Essen'    |

hat\_1

| <u>Person-Id</u> | <u>Adress-Id</u> |
|------------------|------------------|
| 'K001'           | 1                |
| 'M002'           | 2                |
| 'K003'           | 3                |
| 'M004'           | 4                |
| 'M005'           | 5                |
| 'M006'           | 6                |
| 'K007'           | 7                |
| 'M008'           | 8                |

Kunde

| <u>Person-Id</u> |
|------------------|
| 'K001'           |
| 'K003'           |
| 'K005'           |
| 'K007'           |

Mitarbeiter

| <u>Person-Id</u> | <u>Sachgebiet</u>    |
|------------------|----------------------|
| 'M002'           | 'Asienreisen'        |
| 'M004'           | 'Europareisen'       |
| 'M006'           | 'Afrikareisen'       |
| 'M008'           | 'Nordamerika Reisen' |

Reise

| <u>Reise-Id</u> | Teilnehmerzahl | Buchungsdatum                     | Versicherung | Gesamtpreis | <u>Person-Id</u> |
|-----------------|----------------|-----------------------------------|--------------|-------------|------------------|
| 444             | 3              | 2025-02-12<br>17:55:09:804<br>981 | true         | 1530.79     | 'K001'           |
| 445             | 1              | 2025-02-12<br>17:55:09:804<br>981 | false        | 650.5       | 'K003'           |
| 446             | 2              | 2025-02-12<br>17:55:09:804<br>981 | true         | 745.89      | 'K005'           |
| 447             | 2              | 2025-02-12<br>17:55:09:804<br>981 | true         | 459.5       | 'K007'           |

buchen

| <u>Reise-Id</u> | Buchungsstatus   | <u>Person-Id</u> |
|-----------------|------------------|------------------|
| 444             | 'laeuft'         | 'K001'           |
| 445             | 'in Bearbeitung' | 'M002'           |
| 445             | 'in Bearbeitung' | 'K003'           |
| 446             | 'storniert'      | 'K005'           |
| 447             | 'laeuft'         | 'K007'           |

besteht aus

| <u>Reise-Id</u> | <u>Etappen-Id</u> |
|-----------------|-------------------|
| 444             | 301               |
| 444             | 302               |
| 444             | 303               |



|     |     |
|-----|-----|
| 445 | 401 |
| 445 | 402 |
| 446 | 501 |
| 446 | 502 |
| 447 | 601 |

### Etappe

| <u>Etappe-Id</u> | Startdatum   | Enddatum     | Reiseziel   | Etappenkosten | Transfer |
|------------------|--------------|--------------|-------------|---------------|----------|
| 301              | '2025-04-01' | '2025-04-12' | 'Wien'      | 853           | true     |
| 302              | '2025-04-13' | '2025-04-16' | 'Paris'     | 450           | false    |
| 303              | '2025-04-17' | '2025-04-30' | 'Madrid'    | 227.79        | true     |
| 401              | '2025-07-01' | '2025-07-12' | 'Berlin'    | 370.5         | true     |
| 402              | '2025-07-13' | '2025-07-18' | 'London'    | 280           | false    |
| 501              | '2025-06-01' | '2025-06-08' | 'Amsterdam' | 300           | false    |
| 502              | '2025-06-09' | '2025-06-20' | 'Istanbul'  | 445.89        | false    |
| 601              | '2025-07-01' | '2025-07-12' | 'Hanoi'     | 459.5         | true     |

### Land

| <u>Land-Id</u> | Staat                     | Sprache        | Waehrung | Preisklasse |
|----------------|---------------------------|----------------|----------|-------------|
| 'AT'           | 'Oesterreich'             | 'deutsch'      | 'Euro'   | 2           |
| 'FR'           | 'Frankreich'              | 'franzoesisch' | 'Euro'   | 2           |
| 'ES'           | 'Spanien'                 | 'spanisch'     | 'Euro'   | 1           |
| 'DE'           | 'Deutschland'             | 'deutsch'      | 'Euro'   | 2           |
| 'GB'           | 'Vereinigtes Koenigreich' | 'englisch'     | 'Pfund'  | 3           |

|      |               |                       |                            |   |
|------|---------------|-----------------------|----------------------------|---|
| 'NL' | 'Niederlande' | 'niederlaendlich<br>, | 'Euro'                     | 3 |
| 'TR' | 'Tuerkei'     | 'tuerkisch'           | 'Lira'                     | 1 |
| 'VN' | 'Vietnam'     | 'vietnamesisch'       | 'Vietnamesische<br>r Dong' | 1 |

liegt in

| <u>Etappen-Id</u> | <u>Land-Id</u> |
|-------------------|----------------|
| 301               | 'AT'           |
| 302               | 'FR'           |
| 303               | 'ES'           |
| 401               | 'DE'           |
| 402               | 'GB'           |
| 501               | 'NL'           |
| 502               | 'TR'           |
| 601               | 'VN'           |

Unterkunft

| <u>Unterkunfts-<br/>Id</u> | Unterkunftsk<br>osten | Unterkunftsar<br>t | Klassifizierung | Verpflegung | Zimmer |
|----------------------------|-----------------------|--------------------|-----------------|-------------|--------|
| 001                        | 250                   | 'Mietung'          | '3-Sterne'      | false       | 1      |
| 002                        | 280                   | 'Hotel'            | '5-Sterne'      | true        | 1      |
| 003                        | 200                   | 'Mietung'          | '5-Sterne'      | false       | 3      |
| 004                        | 80                    | 'Hotel'            | '1-Stern'       | true        | 3      |
| 005                        | 45                    | 'Mietung'          | '3-Sterne'      | false       | 3      |
| 006                        | 300                   | 'Hostel'           | '2-Sterne'      | false       | 2      |
| 007                        | 200                   | 'Hostel'           | '3-Sterne'      | false       | 2      |

|     |     |         |            |      |   |
|-----|-----|---------|------------|------|---|
| 008 | 360 | 'Hotel' | '4-Sterne' | true | 2 |
|-----|-----|---------|------------|------|---|

gehört\_zu

| <u>Unterkunfts-Id</u> | <u>Etappe-Id</u> |
|-----------------------|------------------|
| 001                   | 401              |
| 002                   | 402              |
| 003                   | 303              |
| 004                   | 301              |
| 005                   | 302              |
| 006                   | 501              |
| 007                   | 502              |
| 008                   | 601              |

Transport

| Transport-Id | Transportmittel | Start        | Ende       | Zwischenstop | Transportkosten |
|--------------|-----------------|--------------|------------|--------------|-----------------|
| 010          | 'Bus'           | '2025-04-01' | 2025-04-01 | 0            | 23.00           |
| 011          | 'Bus'           | '2025-04-16' | 2025-04-17 | 2            | 27.79           |
| 012          | 'Zug'           | '2025-07-01' | 2025-07-01 | 2            | 120.50          |
| 013          | 'Flugzeug'      | '2025-06-09' | 2025-06-09 | 0            | 245.89          |
| 014          | 'Zug'           | '2025-07-01' | 2025-07-01 | 2            | 99.50           |

hat\_2

| <u>Transport-Id</u> | <u>Etappe-Id</u> |
|---------------------|------------------|
| 010                 | 301              |
| 011                 | 303              |
| 012                 | 401              |
| 013                 | 502              |
| 014                 | 601              |

# Das Relationale Modell

Person

| <u>Person-Id</u> | Name         | Vorname   | Email                   | TelefonNr | <u>Adress-Id</u> |
|------------------|--------------|-----------|-------------------------|-----------|------------------|
| 'K001'           | 'Müller'     | 'Klaus'   | 'muellerchen@gmail.com' | 12345     | 1                |
| 'M002'           | 'Mustermann' | 'Max'     | 'm.Mueller@gmail.com'   | 678910    | 2                |
| 'K003'           | 'Kruse'      | 'Otto'    | 'kruOtto@gmail.com'     | 111213    | 3                |
| 'M004'           | 'Lampe'      | 'Hannah'  | 'h.Lampe@gmail.com'     | 141516    | 3                |
| 'K005'           | 'Parker'     | 'Lisa'    | 'LiParker@gmail.com'    | 171819    | 4                |
| 'M006'           | 'Can'        | 'Selina'  | 'S.Can@gmail.com'       | 202123    | 5                |
| 'K007'           | 'Choi'       | 'Kai'     | 'choi.kai@gmail.com'    | 242526    | 6                |
| 'M008'           | 'Lomberg'    | 'Mareike' | 'm.Lomberg@gmail.com'   | 282930    | 7                |

Adresse

| <u>Adress-Id</u> | Straße             | Hausnummer | PLZ   | Ort        |
|------------------|--------------------|------------|-------|------------|
| 1                | 'Knochenstraße'    | '3'        | 24111 | 'Bremen'   |
| 2                | 'Sandstraße'       | '29'       | 29141 | 'Bremen'   |
| 3                | 'Palaststraße'     | '16'       | 20221 | 'Bremen'   |
| 4                | 'Schulstraße'      | '8'        | 22222 | 'Hamburg'  |
| 5                | 'Taunusstraße'     | '7'        | 41321 | 'Muenchen' |
| 6                | 'Spandauer Straße' | '5'        | 33123 | 'Berlin'   |

|   |              |       |       |         |
|---|--------------|-------|-------|---------|
| 7 | 'Baumstraße' | '106' | 52675 | 'Essen' |
|---|--------------|-------|-------|---------|

### Kunde

| <u>Person-Id</u> |
|------------------|
| 'K209'           |
| 'K001'           |
| 'K003'           |
| 'K005'           |
| 'K007'           |

### Mitarbeiter

| <u>Person-Id</u> | Sachgebiet           |
|------------------|----------------------|
| 'M2010'          | 'Unbestimmt'         |
| 'M002'           | 'Asienreisen'        |
| 'M004'           | 'Europareisen'       |
| 'M006'           | 'Afrikareisen'       |
| 'M008'           | 'Nordamerika Reisen' |

### buchen

| <u>Reise-Id</u> | Buchungsstatus   | <u>Person-Id</u> |
|-----------------|------------------|------------------|
| 444             | 'laeuft'         | 'K001'           |
| 444             | 'laeuft'         | 'M002'           |
| 445             | 'in Bearbeitung' | 'M002'           |
| 445             | 'in Bearbeitung' | 'K003'           |
| 446             | 'storniert'      | 'K005'           |

|     |          |        |
|-----|----------|--------|
| 447 | 'laeuft' | 'K007' |
|-----|----------|--------|

### Reise

| <u>Reise-Id</u> | Teilnehmerzahl | Buchungsdatum                     | Versicherung | Gesamtpreis | <u>Person-Id</u> |
|-----------------|----------------|-----------------------------------|--------------|-------------|------------------|
| 444             | 3              | 2025-02-12<br>17:55:09:804<br>981 | true         | 1530.79     | 'K001'           |
| 445             | 1              | 2025-02-12<br>17:55:09:804<br>981 | false        | 650.50      | 'K003'           |
| 446             | 2              | 2025-02-12<br>17:55:09:804<br>981 | true         | 745.89      | 'K005'           |
| 447             | 2              | 2025-02-12<br>17:55:09:804<br>981 | true         | 459.50      | 'K007'           |

### Land

| <u>Land-Id</u> | Staat                        | Sprache               | Waehrung                   | Preisklasse |
|----------------|------------------------------|-----------------------|----------------------------|-------------|
| 'AT'           | 'Oesterreich'                | 'deutsch'             | 'Euro'                     | 2           |
| 'FR'           | 'Frankreich'                 | 'franzoesisch'        | 'Euro'                     | 2           |
| 'ES'           | 'Spanien'                    | 'spanisch'            | 'Euro'                     | 1           |
| 'DE'           | 'Deutschland'                | 'deutsch'             | 'Euro'                     | 2           |
| 'GB'           | 'Vereinigtes<br>Koenigreich' | 'englisch'            | 'Pfund'                    | 3           |
| 'NL'           | 'Niederlande'                | 'niederlaendlich<br>, | 'Euro'                     | 3           |
| 'TR'           | 'Tuerkei'                    | 'tuerkisch'           | 'Lira'                     | 1           |
| 'VN'           | 'Vietnam'                    | 'vietnamesisch'       | 'Vietnamesische<br>r Dong' | 1           |

### Etappe

| <b><u>Etappe</u><br/><u>n-Id</u></b> | <b>Startdat<br/>um</b> | <b>Enddatu<br/>m</b> | <b>Reisezi<br/>el</b> | <b>Etappen<br/>kosten</b> | <b>Transfer</b> | <b><u>Land-Id</u></b> | <b><u>Unterku<br/>nfts-Id</u></b> | <b><u>Reise-Id</u></b> |
|--------------------------------------|------------------------|----------------------|-----------------------|---------------------------|-----------------|-----------------------|-----------------------------------|------------------------|
| 301                                  | '2025-04-01'           | '2025-04-12'         | 'Wien'                | 853                       | true            | 'AT'                  | 4                                 | 444                    |
| 302                                  | '2025-04-13'           | '2025-04-16'         | 'Paris'               | 450                       | false           | 'FR'                  | 5                                 | 444                    |
| 303                                  | '2025-04-17'           | '2025-04-30'         | 'Madrid'              | 227.79                    | true            | 'ES'                  | 3                                 | 444                    |
| 401                                  | '2025-07-01'           | '2025-07-12'         | 'Berlin'              | 370.5                     | true            | 'DE'                  | 1                                 | 445                    |
| 402                                  | '2025-07-13'           | '2025-07-18'         | 'London'              | 280                       | false           | 'GB'                  | 2                                 | 445                    |
| 501                                  | '2025-06-01'           | '2025-06-08'         | 'Amster<br>dam'       | 300                       | false           | 'NL'                  | 6                                 | 446                    |
| 502                                  | '2025-06-09'           | '2025-06-20'         | 'Istanbul'            | 445.89                    | false           | 'TR'                  | 7                                 | 446                    |
| 601                                  | '2025-07-01'           | '2025-07-12'         | 'Hanoi'               | 459.5                     | true            | 'VN'                  | 8                                 | 447                    |

### Transport

| <b><u>Transpor<br/>t-Id</u></b> | <b>Transport<br/>mittel</b> | <b>Start</b> | <b>Ende</b>  | <b>Dauer</b> | <b>Zwischen<br/>stopp</b> | <b>Transport<br/>kosten</b> | <b><u>Etappen-<br/>Id</u></b> |
|---------------------------------|-----------------------------|--------------|--------------|--------------|---------------------------|-----------------------------|-------------------------------|
| 010                             | 'Bus'                       | '2025-04-01' | '2025-04-01' | '04:30:00'   | 0                         | 23.00                       | 301                           |
| 011                             | 'Bus'                       | '2025-04-16' | '2025-04-17' | '06:30:00'   | 2                         | 27.79                       | 303                           |
| 012                             | 'Zug'                       | '2025-07-01' | '2025-07-01' | '07:50:00'   | 2                         | 120.50                      | 401                           |
| 013                             | 'Flugzeu<br>g'              | '2025-06-09' | '2025-06-09' | '05:27:00'   | 0                         | 245.89                      | 502                           |
| 014                             | 'Zug'                       | '2025-07-01' | '2025-07-01' | '06:07:00'   | 2                         | 99.50                       | 601                           |



|  |  |     |     |  |  |  |  |
|--|--|-----|-----|--|--|--|--|
|  |  | 01' | 01' |  |  |  |  |
|--|--|-----|-----|--|--|--|--|

### Unterkunft

| <u>Unterkunfts-Id</u> | Unterkunftskosten | Unterkunftsart | Klassifizierung | Verpflegung | Zimmer |
|-----------------------|-------------------|----------------|-----------------|-------------|--------|
| 001                   | 250               | 'Mietung'      | '3-Sterne'      | false       | 1      |
| 002                   | 280               | 'Hotel'        | '5-Sterne'      | true        | 1      |
| 003                   | 200               | 'Mietung'      | '5-Sterne'      | false       | 3      |
| 004                   | 80                | 'Hotel'        | '1-Stern'       | true        | 3      |
| 005                   | 45                | 'Mietung'      | '3-Sterne'      | false       | 3      |
| 006                   | 300               | 'Hostel'       | '2-Sterne'      | false       | 2      |
| 007                   | 200               | 'Hostel'       | '3-Sterne'      | false       | 2      |
| 008                   | 360               | 'Hotel'        | '4-Sterne'      | true        | 2      |

## Beschreibung

Nach dem Überführen des ER-Modells in die Normalform, kann man es im letzten Schritt ins Relationale Modell überführen. Dabei bleiben die Entitäten-Tabellen bestehen, während einige Relations-Tabellen entfallen, sofern die zugrunde liegende Kardinalität dies erlaubt. Die Tabelle "buchen" bleibt bestehen, da es sich um eine n:m-Beziehung mit zusätzlichen Attributen handelt. Andere Relations-Tabellen können durch Fremdschlüssel in den Entitäten-Tabellen ersetzt werden. Hierfür gibt es eine Ausführliche Beschreibung der Schritte:

#### Tabelle 'hat\_1':

Die Kardinalität zwischen 'Person' und 'Adresse' ist 1:N, und nicht N:M. Dies bedeutet, dass der Primärschlüssel 'Adress-Id' als Fremdschlüssel für die 'Person'-Tabelle verwendet werden kann und eine Zwischentabelle 'hat\_1' nicht notwendig ist.

#### Tabelle 'besteht aus':

Die Beziehung zwischen 'Reise' und 'Etappe' ist 1:N. Aus diesem Grund kann die 'Reise-Id' als Fremdschlüssel für die 'Etappen'-Tabelle benutzt werden. Somit fällt die 'besteht\_aus' Tabelle weg. Wenn eine Reise mehrere Etappen hat, dann lösen wir es indem wir über mehrere Zeilen eine Reise mit verschiedenen Etappen erstellen.

#### Tabelle 'liegt\_in':

Zwischen 'Land' und 'Etappe' herrscht die Kardinalität 1:N, dadurch wird die 'liegt\_in'-Tabelle überflüssig. 'Land' wird ein Fremdschlüssel von 'Etappe', da von 'Land' die Kardinalität 1 aus geht.

#### Tabelle 'gehört\_zu':

Die Kardinalität zwischen 'Etappe' und 'Unterkunft' ist 1:1. Der Primärschlüssel von 'Unterkunft' wird zum Fremdschlüssel von 'Etappe', die Relationstabelle 'gehört\_zu' ist somit nicht notwendig. Da es sich um eine 1:1 Beziehung handelt, könnte auch die 'Etappen-Id' Fremdschlüssel von Unterkunft werden.

#### Tabelle 'hat\_2':

Zwischen 'Etappe' und 'Transport' herrscht die Kardinalität 1:N, dadurch wird die 'hat\_2'-Tabelle überflüssig. 'Etappen-Id' wird ein Fremdschlüssel von 'Transport', da von 'Land' die Kardinalität 1 aus geht.

Das relationale Modell bietet eine klare und strukturierte Darstellung von Daten, indem es Entitäten und ihre Beziehungen in Tabellen abbildet. Nach diesem Schritt ist es möglich, mit einem sauberen und strukturierten Skript zu beginnen.

# Unser Skript

In dieser Dokumentation wird der eigentliche Quellcode nicht direkt aufgeführt, sondern lediglich beschrieben. Der vollständige Code befindet sich in den zugehörigen SQL-Dateien. Die Dokumentation orientiert sich an den Überschriften innerhalb der SQL-Dateien (Reisesystem.sql&Befehle\_Reisesystem.sql). Jeder Abschnitt dieser Dokumentation erläutert die entsprechenden Code Teile und deren Funktion, ohne den vollständigen Code abzubilden. So erhält man eine strukturierte Erklärung der einzelnen Skriptteile, während der eigentliche Code in den SQL-Dateien erhalten bleibt.

## Trigger 1

Diese Triggerfunktion ist zuständig, um das Buchungsdatum bei einem neuen Eintrag einer Reise automatisch zu setzen. Der Trigger wird ausgelöst, wenn man über einen INSERT eine neue Reise anlegt, die in der Reise-Tabelle gespeichert werden soll.

Es wird auf die aktuelle Zeit (`CURRENT_TIMESTAMP`) bzw. die Laufzeit gesetzt. Dabei passiert dies, mithilfe der `FOR EACH ROW` Schleife, für jede Zeile.

Der Trigger wird vor dem Einfügen (`BEFORE INSERT`) ausgeführt. Dadurch wird das Buchungsdatum gesetzt, bevor der Datensatz in der Datenbank gespeichert wird

## Trigger 2

Der Trigger zur Fallunterscheidung der Person-Id beim INSERT einer neuen Person ausgelöst. Dabei wird überprüft, ob die Person\_Id mit "M" (%M für Mitarbeiter) oder "K" (%K für Kunde) beginnt.

Hat die Person-Id eines neuen Eintrags ein "K" am Anfang, wird die Person automatisch in die Kunden-Tabelle und nicht nur in die Person-Tabelle übernommen. Da Kunden keine zusätzlichen Attribute besitzen, enthält diese Tabelle lediglich die Person-Ids.

Beginnt die Person-Id mit "M", wird die Person in die Mitarbeiter-Tabelle eingefügt. Zusätzlich wird das Attribut Sachgebiet mit dem Standardwert "Unbestimmt" initialisiert, das später aktualisiert werden muss.

Durch die `insert_customer_or_employee()` Funktion wird sichergestellt, dass jede neue Person direkt in der richtigen Tabelle gespeichert wird, ohne dass eine manuelle Zuordnung erforderlich ist.

Der Trigger wird mit `AFTER INSERT` definiert, was bedeutet, dass die Eintragung in die Person-Tabelle zuerst vollständig abgeschlossen sein muss, bevor die zusätzliche Verarbeitung erfolgt. Dadurch wird sichergestellt, dass die Person-Id bereits existiert und referenziert werden kann, bevor sie in die Kunden- oder Mitarbeiter-Tabelle eingefügt wird.

## Trigger 3

Dieser Trigger sorgt dafür, dass die 'Etappenkosten' aktualisiert werden, wenn sich Daten in der Tabelle 'Transport' ändern (durch INSERT, UPDATE oder DELETE).

Es wird eine Summe der Transportkosten für die betroffene Etappe (basierend auf 'Etappen\_Id' in der Tabelle 'Transport') berechnet. Zudem werden die 'Unterkunftskosten' für die betroffene Etappe (basierend auf 'Unterkunfts\_Id' in der Tabelle 'Unterkunft') abgerufen. Der Wert für 'Etappenkosten' wird dann als Summe der Transportkosten und Unterkunftskosten gesetzt. Falls es keine Transport- oder Unterkunftskosten gibt, wird mit 0 als Standardwert gerechnet (durch Verwendung von COALESCE()).

## Trigger 4

Bei diesem Trigger werden die 'Etappenkosten' in der Tabelle 'Etappe' aktualisiert, wenn sich Daten in der Tabelle 'Unterkunft' ändern.

Auch hier wird die Summe der Transportkosten abgerufen, sowie der Wert der Unterkunftskosten. Die neuen Etappenkosten werden als Summe aus den Transportkosten und Unterkunftskosten gesetzt. COALESCE() wird verwendet, um sicherzustellen, dass fehlende Werte durch 0 ersetzt werden.

## Trigger 5

In diesem Trigger werden die 'Etappenkosten' aktualisiert, wenn eine neue Unterkunft hinzugefügt wird.

Hierfür erstellen wir eine Funktion, die die Operation ausführt, die wir im Trigger haben wollen. Dabei definieren wir in der Funktion, dass wir die Tabelle 'Etappe' aktualisieren wollen. Dann geben wir mit 'SET' die betroffene Spalte an, die aktualisiert werden soll. Mit 'SELECT' geben wir an, aus welchen Tabellen die Kosten zusammengerechnet werden. Wir addieren die Summe von 'Transportkosten' und 'Unterkunftskosten', der 'Etappen-Id' zusammen, die in diesen Tabellen steht. Wenn es keine 'Transportkosten' und/oder 'Unterkunftskosten' gibt, wird dann 0 zurückgegeben. Zuletzt in der Funktion gehen wir sicher mit 'WHERE', dass wir nur die gerade eingefügte Etappe aktualisieren und mit 'RETURN NEW' repräsentiert die gerade eingefügte Zeile, die nach der Ausführung der Funktion immer noch verfügbar ist.

Dann erstellen wir den Trigger und geben an, dass der Trigger nur ausgeführt werden soll, wenn etwas in 'Etappe' hinzugefügt wird. Zum Schluss wird mit 'EXECUTE FUNCTION' die erstellt Funktion aufgerufen, um die 'Etappenkosten' zu aktualisieren.

## Trigger 6

Dieser Trigger aktualisiert die 'Gesamtpreis' Spalte aus 'Reise', wenn sich die 'Etappenkosten' einer Etappe ändern.

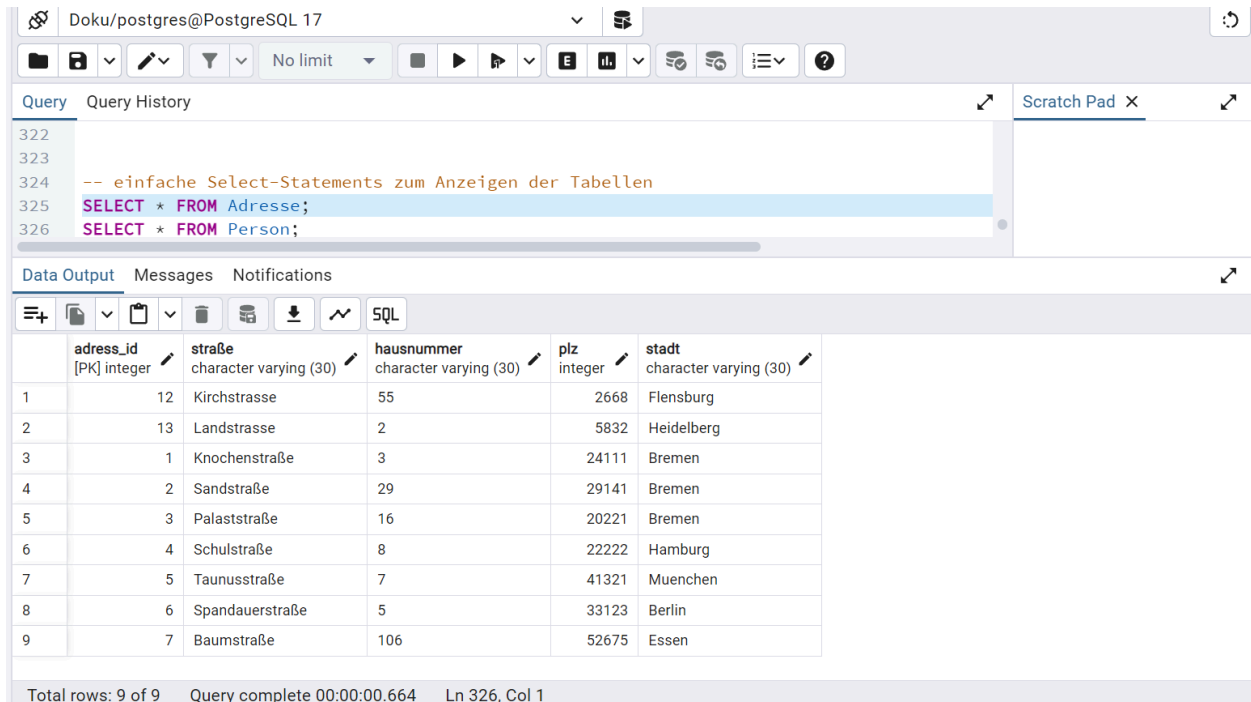
Zunächst erstellen wir eine Funktion, die die 'Gesamtpreis' aktualisieren soll. Hierzu wird mit 'UPDATE' die Tabelle 'Reise' aktualisiert. Mit 'SET' geben wir an, dass es in der Spalte 'Gesamtpreis' geschehen soll. Mit 'SELECT SUM' wird die Summe der betroffenen Reise berechnet. Mit 'WHERE' stellt man sicher, dass die betreffende Reise aktualisiert wird.

Dann wird der Trigger erstellt, der dann auslöst, wenn sich 'Etappenkosten' aktualisiert. Mit 'EXECUTE FUNCTION' wird dann die erstellte Funktion aufgerufen.

# Testszenarios mit SQL

## Einfache Select-Statements

Das erste SELECT Statement ist das der Adresse-Tabelle:



The screenshot shows a PostgreSQL client interface with a query editor and a data output window. The query editor contains the following SQL code:

```
-- einfache Select-Statements zum Anzeigen der Tabellen
SELECT * FROM Adresse;
SELECT * FROM Person;
```

The data output window displays the results of the first query, showing 9 rows of data from the 'Adresse' table. The columns are: **adress\_id** [PK] integer, **straße** character varying (30), **hausnummer** character varying (30), **plz** integer, and **stadt** character varying (30).

|   | adress_id [PK] integer | straße character varying (30) | hausnummer character varying (30) | plz integer | stadt character varying (30) |
|---|------------------------|-------------------------------|-----------------------------------|-------------|------------------------------|
| 1 | 12                     | Kirchstrasse                  | 55                                | 2668        | Flensburg                    |
| 2 | 13                     | Landstrasse                   | 2                                 | 5832        | Heidelberg                   |
| 3 | 1                      | Knochenstraße                 | 3                                 | 24111       | Bremen                       |
| 4 | 2                      | Sandstraße                    | 29                                | 29141       | Bremen                       |
| 5 | 3                      | Palaststraße                  | 16                                | 20221       | Bremen                       |
| 6 | 4                      | Schulstraße                   | 8                                 | 22222       | Hamburg                      |
| 7 | 5                      | Taunusstraße                  | 7                                 | 41321       | Muenchen                     |
| 8 | 6                      | Spandauerstraße               | 5                                 | 33123       | Berlin                       |
| 9 | 7                      | Baumstraße                    | 106                               | 52675       | Essen                        |

Total rows: 9 of 9 Query complete 00:00:00.664 Ln 326, Col 1

In unserem Skript haben wir insgesamt zehn Personen erstellt, doch es gibt nur neun unterschiedliche Adress-Ids, dies bedeutet, dass eine Adress-Id von zwei Personen gemeinsam genutzt wird.

Als nächstes haben wir das SELECT Statement für die Person-Tabelle:

Query

Query History

Scratch Pad

324

-- einfache Select-Statements zum Anzeigen der Tabellen

325

SELECT \* FROM Adresse;

326

SELECT \* FROM Person;

Data Output

Messages

Notifications

SQL

|    | person_id<br>[PK] character varying (10) | name<br>character varying (30) | vorname<br>character varying (30) | email<br>character varying (30) | telefonnr<br>integer | adress_id<br>integer |
|----|------------------------------------------|--------------------------------|-----------------------------------|---------------------------------|----------------------|----------------------|
| 1  | K209                                     | Schoeneberg                    | Valentina                         | vali@gmail.com                  | 777777               | 12                   |
| 2  | M2010                                    | Schmidt                        | Peter                             | p.schmidt@gmail.com             | 88888                | 13                   |
| 3  | K001                                     | Mueller                        | Klaus                             | muellerchen@gmail.com           | 12345                | 1                    |
| 4  | M002                                     | Mustermann                     | Max                               | m.Mueller@gmail.com             | 678910               | 2                    |
| 5  | K003                                     | Kruse                          | Otto                              | kruOtto@gmail.com               | 111213               | 3                    |
| 6  | M004                                     | Lampe                          | Hannah                            | h.Lampe@gmail.com               | 141516               | 3                    |
| 7  | K005                                     | Parker                         | Lisa                              | LiParker@gmail.com              | 171819               | 4                    |
| 8  | M006                                     | Can                            | Selina                            | S.Can@gmail.com                 | 202123               | 5                    |
| 9  | K007                                     | Choi                           | Kai                               | choi.kai@gmail.com              | 242526               | 6                    |
| 10 | M008                                     | Lomberg                        | Mareike                           | m.Lomberg@gmail.com             | 282930               | 7                    |

Wie erwähnt, haben wir zehn Personen hinzugefügt. Dabei haben die Personen “K003” und “M004” dieselbe Adress-Id (3). Dies verdeutlicht die 1:N-Kardinalität zwischen Person und Adresse, da mehrere Personen sich eine Adresse teilen können. Außerdem erkennt man, dass die Person-Tabelle alle Personen enthält, unabhängig davon, ob es sich um Kunden oder Mitarbeiter handelt.

Die spezifische Tabelle der Kunden sieht wie folgt aus:

Help

Properties X SQL X Statistics X Dependencies X Dependents X Processes X

Doku/postgres@PostgreSQL 17\* X

</

Die fünf erstellten Kunden mit ihrer Person-Id werden in dieser Tabelle gespeichert. Da der Kunde von Person erbt und keine zusätzlichen Attribute besitzt, enthält die Tabelle lediglich eine Referenz über die Person-Id.

Im Gegensatz dazu besitzen Mitarbeiter zusätzlich das Attribut Sachgebiet:

The screenshot shows a PostgreSQL IDE interface. The top menu bar includes Properties, SQL, Statistics, Dependencies, Dependents, Processes, and a connection tab for 'Doku/postgres@PostgreSQL 17\*'. Below the menu is a toolbar with icons for file operations, query execution, and settings. The main window is divided into two panes. The left pane, titled 'Query', contains a SQL script with five lines: `SELECT * FROM Adresse;`, `SELECT * FROM Person;`, `SELECT * FROM Kunden;`, `SELECT * FROM Mitarbeiter;`, and `SELECT * FROM Land;`. The fourth line is highlighted. The right pane, titled 'Data Output', displays the results of the selected query in a table format. The table has two columns: 'person\_id' (PK) character varying (10) and 'sachgebiet' character varying (30). It contains five rows of data, all with 'Unbestimmt' in the 'sachgebiet' column. At the bottom of the interface, a green status bar indicates: 'Successfully run. Total query runtime: 278 msec. 5 rows affected.'

|   | person_id<br>[PK] character varying (10) | sachgebiet<br>character varying (30) |
|---|------------------------------------------|--------------------------------------|
| 1 | M2010                                    | Unbestimmt                           |
| 2 | M002                                     | Unbestimmt                           |
| 3 | M004                                     | Unbestimmt                           |
| 4 | M006                                     | Unbestimmt                           |
| 5 | M008                                     | Unbestimmt                           |

Das Sachgebiet ist zunächst bei allen hinzugefügten Mitarbeitern auf “Unbestimmt” gesetzt. Es war schwer umsetzbar, dieses Attribut direkt über den Trigger zuzuweisen. Aus diesem Grund wird das Sachgebiet über einen UPDATE Befehl hinzufügen.

Die in unserer Datenbank gespeicherten Länder sind in der folgenden Tabelle zu sehen. Der Output stammt aus einem SELECT-Statement auf die Länder-Tabelle:



Properties × SQL × Statistics × Dependencies × Dependents × Processes × Doku/postgres@PostgreSQL 17\* ×

Doku/postgres@PostgreSQL 17

Query Query History Scratch Pad ×

```

325 SELECT * FROM Adresse;
326 SELECT * FROM Person;
327 SELECT * FROM Kunden;
328 SELECT * FROM Mitarbeiter;
329 SELECT * FROM Land;

```

Data Output Messages Notifications

|   | land_id<br>[PK] character varying (2) | staat<br>character varying (30) | sprache<br>character varying (30) | waehrung<br>character varying (30) | preisklasse<br>integer |
|---|---------------------------------------|---------------------------------|-----------------------------------|------------------------------------|------------------------|
| 1 | AT                                    | Oesterreich                     | deutsch                           | Euro                               | 2                      |
| 2 | FR                                    | Frankreich                      | franzoesisch                      | Euro                               | 2                      |
| 3 | ES                                    | Spanien                         | spanisch                          | Euro                               | 1                      |
| 4 | DE                                    | Deutschland                     | deutsch                           | Euro                               | 2                      |
| 5 | GB                                    | Vereinigtes Koenigreich         | englisch                          | Pfund                              | 3                      |
| 6 | NL                                    | Niederlande                     | niederlaendisch                   | Euro                               | 3                      |
| 7 | TR                                    | Tuerkei                         | tuerkisch                         | Lira                               | 1                      |
| 8 | VN                                    | Vietnam                         | vietnamesisch                     | Vietnamesischer Dong               | 1                      |

Total rows: 8 of 8 Query complete 00:00:00.302 Ln 330 Col 1

Der Datensatz enthält insgesamt acht verschiedene Länder aus zwei unterschiedlichen Kontinenten. Zudem ist für jede Preisklasse (1–3, basierend auf den Aufenthaltskosten) mindestens ein repräsentatives Land enthalten.

Die Etappen-Tabelle sieht wie folgt aus:

SQL Statistics Dependencies Dependents Processes Doku/postgres@P... Doku2/postgres@PostgreSQL 17\*

Doku2/postgres@PostgreSQL 17

Query Query History Scratch Pad

```

311 SELECT * FROM Etappe;
312 SELECT * FROM Reise;
313 SELECT * FROM buchen;
314 SELECT * FROM Unterkunft;
315 SELECT * FROM Transport;

```

Data Output Messages Notifications

|   | etappen_id<br>[PK] integer | startdatum<br>date | enddatum<br>date | etappenkosten<br>double precision | reiseziel<br>character varying (30) | transfer<br>boolean | land_id<br>character varying (2) | unterkunts_id<br>integer | reise_id<br>integer |
|---|----------------------------|--------------------|------------------|-----------------------------------|-------------------------------------|---------------------|----------------------------------|--------------------------|---------------------|
| 1 | 302                        | 2025-04-13         | 2025-04-16       | 450                               | Paris                               | false               | FR                               | 5                        | 444                 |
| 2 | 402                        | 2025-07-13         | 2025-07-18       | 280                               | London                              | false               | GB                               | 2                        | 445                 |
| 3 | 501                        | 2025-06-01         | 2025-06-08       | 300                               | Amsterdam                           | false               | NL                               | 6                        | 446                 |
| 4 | 301                        | 2025-04-01         | 2025-04-12       | 853                               | Wien                                | true                | AT                               | 4                        | 444                 |
| 5 | 303                        | 2025-04-17         | 2025-04-30       | 227.79                            | Madrid                              | true                | ES                               | 3                        | 444                 |
| 6 | 401                        | 2025-07-01         | 2025-07-12       | 370.5                             | Berlin                              | true                | DE                               | 1                        | 445                 |
| 7 | 502                        | 2025-06-09         | 2025-06-20       | 445.89                            | Istanbul                            | false               | TR                               | 7                        | 446                 |
| 8 | 601                        | 2025-07-01         | 2025-07-12       | 459.5                             | Hanoi                               | true                | VN                               | 8                        | 447                 |

Total rows: 8 of 8 Query complete 00:00:00.288 Ln 312, Col 1

Erkennbar wird, dass die Etappenkosten, aufgrund des Triggers, berechnet sind und die Verknüpfung zur Unterkunfts-Tabelle und Reise-Tabelle (zusammengesetzter Fremdschlüssel unterkunts\_id&reise\_id).

Unsere Datenbank besteht aus acht Etappen und vier Reisen:

Properties X SQL X Statistics X Dependencies X Dependents X Processes X Doku/postgres@PostgreSQL 17\* X

Doku/postgres@PostgreSQL 17

Query Query History Scratch Pad X

```

330 SELECT * FROM Etappe;
331 SELECT * FROM Reise;
332 SELECT * FROM buchen;
333 SELECT * FROM Unterkunft;
334 SELECT * FROM Transport;

```

Data Output Messages Notifications

|   | reise_id<br>[PK] integer | teilnehmerzahl<br>integer | buchungsdatum<br>timestamp without time zone | versicherung<br>boolean | gesamtpreis<br>double precision | person_id<br>character varying (30) |
|---|--------------------------|---------------------------|----------------------------------------------|-------------------------|---------------------------------|-------------------------------------|
| 1 | 444                      | 3                         | 2025-02-12 17:55:09.804981                   | true                    | 1530.79                         | K001                                |
| 2 | 445                      | 1                         | 2025-02-12 17:55:09.804981                   | false                   | 650.5                           | K003                                |
| 3 | 446                      | 2                         | 2025-02-12 17:55:09.804981                   | true                    | 745.89                          | K005                                |
| 4 | 447                      | 2                         | 2025-02-12 17:55:09.804981                   | true                    | 459.5                           | K007                                |

✓ Successfully run. Total query runtime: 308 msec. 4 rows affected. ✕

Total rows: 4 of 4 Query complete 00:00:00.308 Ln 332, Col 1

Genau wie bei der Etappen-Tabelle wurden hier die Gesamtkosten der Reise (die sich aus den Etappenkosten zusammenrechnen) aufgrund des Triggers bereits errechnet. Als Buchungsdatum ist das aktuelle Datum und die aktuelle Zeit für jede Reise erkennbar. Um die Reisen mit den Kunden zu verknüpfen ist die Person-Id mit integriert. Daraus kann man ableiten, dass nur vier Kunden eine Reise gebucht haben.

Insgesamt haben wir also folgende Reisen mit den entsprechenden Etappen:

- Reise 444 mit Etappen: 302(Paris),301(Wien),303(Madrid)
- Reise 445 mit Etappen: 402(London), 401(Berlin)
- Reise 446 mit Etappen: 501(Amsterdam),501(Istanbul)
- Reise 447 mit nur einer Etappe: 601(Hanoi)

Den Überblick über die gebuchten Reisen sieht man in der "buchen" Tabelle:

Properties X SQL X Statistics X Dependencies X Dependents X Processes X Doku/postgres@PostgreSQL 17\* X

Doku/postgres@PostgreSQL 17

Query Query History Scratch Pad X

```

330 SELECT * FROM Etappe;
331 SELECT * FROM Reise;
332 SELECT * FROM buchen;
333 SELECT * FROM Unterkunft;
334 SELECT * FROM Transport;

```

Data Output Messages Notifications

|   | reise_id<br>[PK] integer | buchungsstatus<br>character varying (15) | person_id<br>[PK] character varying (30) |
|---|--------------------------|------------------------------------------|------------------------------------------|
| 1 | 444                      | laeuft                                   | K001                                     |
| 2 | 444                      | laeuft                                   | M002                                     |
| 3 | 445                      | in_Bearbeitung                           | K003                                     |
| 4 | 445                      | in_Bearbeitung                           | M002                                     |
| 5 | 446                      | storniert                                | K005                                     |
| 6 | 447                      | laeuft                                   | K007                                     |

Total rows: 6 of 6 Query complete 00:00:00.290 Ln 333. Col 1

Zwei Reisen wurden mit einem Mitarbeiter zusammen gebucht und die zwei weiteren Reisen wurden eigenständig von den Kunden gebucht. In dieser Tabelle sieht man zusätzlich den Buchungsstatus.

Zu den acht Etappen im Datensatz gibt es jeweils acht Unterkünfte, welches die Kardinalität 1:1 zwischen Etappe und Unterkunft widerspiegelt:

Properties X SQL X Statistics X Dependencies X Dependents X Processes X Doku/postgres@PostgreSQL 17\* X

Doku/postgres@PostgreSQL 17

Query Query History

```

330 SELECT * FROM Etappe;
331 SELECT * FROM Reise;
332 SELECT * FROM buchen;
333 SELECT * FROM Unterkunft;
334 SELECT * FROM Transport;

```

Data Output Messages Notifications

|   | unterkunts_id<br>[PK] integer | unterkuntskosten<br>double precision | unterkuntsart<br>character varying (30) | klassifizierung<br>character varying (8) | verpflegung<br>boolean | zimmer<br>integer |
|---|-------------------------------|--------------------------------------|-----------------------------------------|------------------------------------------|------------------------|-------------------|
| 1 | 1                             | 250                                  | Mietung                                 | 3-Sterne                                 | false                  | 1                 |
| 2 | 2                             | 280                                  | Hotel                                   | 5-Sterne                                 | true                   | 1                 |
| 3 | 3                             | 200                                  | Mietung                                 | 2-Sterne                                 | false                  | 3                 |
| 4 | 4                             | 830                                  | Hotel                                   | 1-Stern                                  | true                   | 3                 |
| 5 | 5                             | 450                                  | Mietung                                 | 3-Sterne                                 | false                  | 3                 |
| 6 | 6                             | 300                                  | Hostel                                  | 2-Sterne                                 | false                  | 2                 |
| 7 | 7                             | 200                                  | Hostel                                  | 3-Sterne                                 | false                  | 2                 |
| 8 | 8                             | 360                                  | Hotel                                   | 4-Sterne                                 | true                   | 2                 |

Total rows: 8 of 8 Query complete 00:00:00.259 Ln 334, Col 1

Zu den acht Etappen gibt es nur fünf eingetragene Transporte, somit gibt es auch Kunden, die den Transport selbst organisieren.

Properties X SQL X Statistics X Dependencies X Dependents X Processes X Doku/postgres@PostgreSQL 17\* X

Doku/postgres@PostgreSQL 17

Query Query History

```

330 SELECT * FROM Etappe;
331 SELECT * FROM Reise;
332 SELECT * FROM buchen;
333 SELECT * FROM Unterkunft;
334 SELECT * FROM Transport;

```

Data Output Messages Notifications

|   | transport_id<br>[PK] integer | transportmittel<br>character varying | start<br>date | ende<br>date | dauer<br>time without time zone | zwischenstopp<br>integer | transportkosten<br>double precision | etappen_id<br>integer |
|---|------------------------------|--------------------------------------|---------------|--------------|---------------------------------|--------------------------|-------------------------------------|-----------------------|
| 1 | 10                           | Bus                                  | 2025-04-01    | 2025-04-01   | 04:30:00                        | 0                        | 23                                  | 301                   |
| 2 | 11                           | Bus                                  | 2025-04-16    | 2025-04-17   | 06:30:00                        | 2                        | 27.79                               | 303                   |
| 3 | 12                           | Zug                                  | 2025-07-01    | 2025-07-01   | 07:50:00                        | 2                        | 120.5                               | 401                   |
| 4 | 13                           | Flugzeug                             | 2025-06-09    | 2025-06-09   | 05:27:00                        | 0                        | 245.89                              | 502                   |
| 5 | 14                           | Zug                                  | 2025-07-01    | 2025-07-01   | 06:07:00                        | 2                        | 99.5                                | 601                   |

## Updates

Um den Mitarbeitern ihre jeweiligen Sachgebiete zuzuweisen, wird ein UPDATE-Befehl verwendet.

Durch die Aktualisierung der Mitarbeiter-Tabelle erhalten die Mitarbeiter spezifische Sachgebiete. Nur der Mitarbeiter "M2010" behält weiterhin den Standardwert. Nach der Ausführung des Updates sind somit alle, bis auf einen Mitarbeiter (der beispielsweise noch eingearbeitet werden muss), einem bestimmten Sachgebiet zugeordnet.

The screenshot shows a PostgreSQL client interface with the following components:

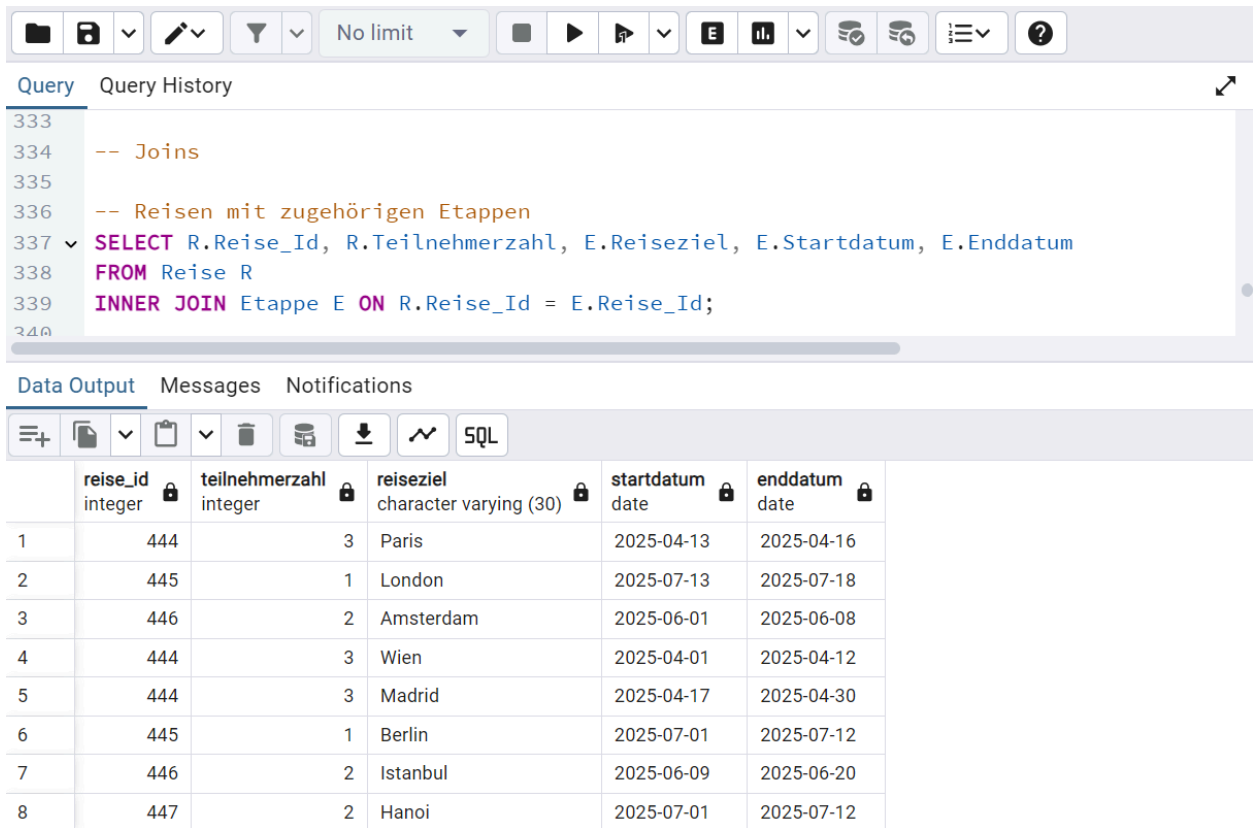
- Query Editor:** Contains an SQL update query:

```
320 SET Sachgebiet = CASE
321     WHEN Person_Id = 'M002' THEN 'Asienreisen'
322     WHEN Person_Id = 'M004' THEN 'Europareise'
323     WHEN Person_Id = 'M006' THEN 'Afrikareisen'
324     WHEN Person_Id = 'M008' THEN 'Nordamerikareisen'
325     ELSE Sachgebiet -- Falls keine Übereinstimmung gefunden wird, bleibt der Wert unver
326 END
327 WHERE Person_Id IN ('M002', 'M004', 'M006', 'M008');
328
329 SELECT * FROM Mitarbeiter;
```
- Data Output:** Displays the results of the query in a table:

|   | person_id<br>[PK] character varying (10) | sachgebiet<br>character varying (30) |
|---|------------------------------------------|--------------------------------------|
| 1 | M2010                                    | Unbestimmt                           |
| 2 | M002                                     | Asienreisen                          |
| 3 | M004                                     | Europareise                          |
| 4 | M006                                     | Afrikareisen                         |
| 5 | M008                                     | Nordamerikareisen                    |
- Status Bar:** Shows "Total rows: 5 of 5", "Query complete 00:00:00.319", and "Ln 328, Col 1".

## Joins

Joins werden genutzt, um Daten aus mehreren Tabellen zu kombinieren.



```
333
334 -- Joins
335
336 -- Reisen mit zugehörigen Etappen
337 SELECT R.Reise_Id, R.Teilnehmerzahl, E.Reiseziel, E.Startdatum, E.Enddatum
338 FROM Reise R
339 INNER JOIN Etappe E ON R.Reise_Id = E.Reise_Id;
```

|   | reise_id<br>integer | teilnehmerzahl<br>integer | reiseziel<br>character varying (30) | startdatum<br>date | enddatum<br>date |
|---|---------------------|---------------------------|-------------------------------------|--------------------|------------------|
| 1 | 444                 | 3                         | Paris                               | 2025-04-13         | 2025-04-16       |
| 2 | 445                 | 1                         | London                              | 2025-07-13         | 2025-07-18       |
| 3 | 446                 | 2                         | Amsterdam                           | 2025-06-01         | 2025-06-08       |
| 4 | 444                 | 3                         | Wien                                | 2025-04-01         | 2025-04-12       |
| 5 | 444                 | 3                         | Madrid                              | 2025-04-17         | 2025-04-30       |
| 6 | 445                 | 1                         | Berlin                              | 2025-07-01         | 2025-07-12       |
| 7 | 446                 | 2                         | Istanbul                            | 2025-06-09         | 2025-06-20       |
| 8 | 447                 | 2                         | Hanoi                               | 2025-07-01         | 2025-07-12       |

Dieser Join gibt alle Reisen zusammen mit ihren zugehörigen Etappen aus. Die Verbindung erfolgt über die Spalte 'Reise\_Id', die sowohl in der Reise- als auch in der Etappe-Tabelle existiert. INNER JOIN bedeutet, dass nur Reisen angezeigt werden, die mindestens eine zugehörige Etappe haben. Die Ausgabe enthält: die Reise\_Id und Teilnehmerzahl aus 'Reise', sowie das Reiseziel und Start- und Enddatum aus 'Etappe'.







Query Query History

```

350
351 -- alle Etappen und ihre Transportmittel und Transportkosten, auch wenn kein Transport
352 v SELECT E.Etappen_Id, E.Reiseziel, T.Transportmittel, T.Transportkosten
353 FROM Etappe E
354 LEFT JOIN Transport T ON E.Etappen_Id = T.Etappen_Id;
355

```

Data Output Messages Notifications

|   | etappen_id<br>integer | reiseziel<br>character varying (30) | transportmittel<br>character varying | transportkosten<br>double precision |
|---|-----------------------|-------------------------------------|--------------------------------------|-------------------------------------|
| 1 | 301                   | Wien                                | Bus                                  | 23                                  |
| 2 | 303                   | Madrid                              | Bus                                  | 27.79                               |
| 3 | 401                   | Berlin                              | Zug                                  | 120.5                               |
| 4 | 502                   | Istanbul                            | Flugzeug                             | 245.89                              |
| 5 | 601                   | Hanoi                               | Zug                                  | 99.5                                |
| 6 | 302                   | Paris                               | [null]                               | [null]                              |
| 7 | 501                   | Amsterdam                           | [null]                               | [null]                              |
| 8 | 402                   | London                              | [null]                               | [null]                              |

Alle Etappen zusammen mit den Transportmitteln und deren Kosten werden angezeigt. Ein LEFT JOIN wird verwendet, um alle Etappen zu zeigen, auch wenn sie keinen Transport haben. Ausgegeben werden: die Etappen\_Id und das Reiseziel aus der Tabelle 'Etappe' und Transportmittel, Transportkosten aus der Tabelle 'Transport'. Falls für eine Etappe kein Transport existiert, so sind die letzten zwei Spalten NULL.

## Löschen

Ein weiteres Testszenario ist das Löschen einer Person. Hierbei wird mit 'ON DELETE CASCADE' die zu löschende Person in allen Tabellen gelöscht, in der sie Einträge hat. Hierfür schauen wir uns die Tabelle an, in der die Person steht. Beispielhaft an Kunde 'K003'. Hier die 'Kunden'-Tabelle:

```

306 -- Löschen einer Person
307 -- vor dem Löschen
308 --SELECT * FROM Person WHERE Person_Id = 'K003';
309 SELECT * FROM Kunden WHERE Person_Id = 'K003';
310 SELECT * FROM Reise WHERE Person_Id = 'K003';
311 --SELECT * FROM buchen WHERE Person_Id = 'K003';
312 -- Löschen
313 DELETE FROM Person

```

Data Output Messages Notifications

|   | person_id<br>[PK] character varying (10) |
|---|------------------------------------------|
| 1 | K003                                     |

Und hier die 'Reise'-Tabelle:

```

306 -- Löschen einer Person
307 -- vor dem Löschen
308 --SELECT * FROM Person WHERE Person_Id = 'K003';
309 SELECT * FROM Kunden WHERE Person_Id = 'K003';
310 SELECT * FROM Reise WHERE Person_Id = 'K003';
311 --SELECT * FROM buchen WHERE Person_Id = 'K003';
312 -- Löschen
313 DELETE FROM Person

```

Data Output Messages Notifications

|   | reise_id<br>[PK] integer | teilnehmerzahl<br>integer | buchungsdatum<br>timestamp without time zone | versicherung<br>boolean | gesamtpreis<br>double precision | person_id<br>character varying (30) |
|---|--------------------------|---------------------------|----------------------------------------------|-------------------------|---------------------------------|-------------------------------------|
| 1 | 445                      | 1                         | 2025-02-12 19:47:07.535827                   | false                   | 650.5                           | K003                                |

Im nächsten Schritt löschen wie diesen Kunden mit dem Befehl 'DELETE':

```

306 -- Löschen einer Person
307 -- vor dem Löschen
308 --SELECT * FROM Person WHERE Person_Id = 'K003';
309 SELECT * FROM Kunden WHERE Person_Id = 'K003';
310 SELECT * FROM Reise WHERE Person_Id = 'K003';
311 --SELECT * FROM buchen WHERE Person_Id = 'K003';
312 -- Löschen
313 DELETE FROM Person
314 WHERE Person_Id = 'K003';

```

Data Output Messages Notifications

DELETE 1

Query returned successfully in 105 msec.

Hiernach überprüfen wir, ob der Kunde von den Tabellen gelöscht wurde. Hier die 'Kunden'-Tabellen:

```

306 -- Löschen einer Person
307 -- vor dem Löschen
308 --SELECT * FROM Person WHERE Person_Id = 'K003';
309 SELECT * FROM Kunden WHERE Person_Id = 'K003';
310 SELECT * FROM Reise WHERE Person_Id = 'K003';
311 --SELECT * FROM buchen WHERE Person_Id = 'K003';
312 -- Löschen
313 ▼ DELETE FROM Person
314 WHERE Person_Id = 'K003';
315 -- nach dem Löschen
316 --SELECT * FROM Person WHERE Person_Id = 'K003';
317 SELECT * FROM Kunden WHERE Person_Id = 'K003';
318 SELECT * FROM Reise WHERE Person_Id = 'K003';
319 --SELECT * FROM buchen WHERE Person_Id = 'K003';

```

Data Output Messages Notifications



person\_id  
[PK] character varying (10)

Und hier die 'Reise'-Tabelle:

```

306 -- Löschen einer Person
307 -- vor dem Löschen
308 --SELECT * FROM Person WHERE Person_Id = 'K003';
309 SELECT * FROM Kunden WHERE Person_Id = 'K003';
310 SELECT * FROM Reise WHERE Person_Id = 'K003';
311 --SELECT * FROM buchen WHERE Person_Id = 'K003';
312 -- Löschen
313 ▼ DELETE FROM Person
314 WHERE Person_Id = 'K003';
315 -- nach dem Löschen
316 --SELECT * FROM Person WHERE Person_Id = 'K003';
317 SELECT * FROM Kunden WHERE Person_Id = 'K003';
318 SELECT * FROM Reise WHERE Person_Id = 'K003';
319 --SELECT * FROM buchen WHERE Person_Id = 'K003';

```

Data Output Messages Notifications



|                          |                           |                                              |                         |                                 |                                     |
|--------------------------|---------------------------|----------------------------------------------|-------------------------|---------------------------------|-------------------------------------|
| reise_id<br>[PK] integer | teilnehmerzahl<br>integer | buchungsdatum<br>timestamp without time zone | versicherung<br>boolean | gesamtpreis<br>double precision | person_id<br>character varying (30) |
|--------------------------|---------------------------|----------------------------------------------|-------------------------|---------------------------------|-------------------------------------|

## Aggregat

Mit den Aggregat-Funktionen kann man Mittelwert, Anzahl, Maximalwert, Minimalwert und Summe aller Tupel anzeigen lassen. Hierfür haben wir folgende Testszenarios getestet.

Den Mittelwert der Teilnehmerzahl:

|    |                                                 |
|----|-------------------------------------------------|
| 59 | -- Aggregat                                     |
| 60 | SELECT AVG (Teilnehmerzahl) FROM Reise;         |
| 61 | SELECT COUNT (*) FROM Reise;                    |
| 62 | SELECT MAX (Unterkunftskosten) FROM Unterkunft; |
| 63 | SELECT MIN (Transportkosten) FROM Transport;    |
| 64 | SELECT SUM (Etappenkosten) FROM Etappe;         |

| Data Output                              | Messages           | Notifications |
|------------------------------------------|--------------------|---------------|
| <div> <div>+</div> <div>SQL</div> </div> |                    |               |
|                                          | avg                | numeric       |
| 1                                        | 2.3333333333333333 |               |

Anzahl aller bestehenden Reisen:

|    |                                                 |
|----|-------------------------------------------------|
| 59 | -- Aggregat                                     |
| 60 | SELECT AVG (Teilnehmerzahl) FROM Reise;         |
| 61 | SELECT COUNT (*) FROM Reise;                    |
| 62 | SELECT MAX (Unterkunftskosten) FROM Unterkunft; |
| 63 | SELECT MIN (Transportkosten) FROM Transport;    |
| 64 | SELECT SUM (Etappenkosten) FROM Etappe;         |

| Data Output                              | Messages | Notifications |
|------------------------------------------|----------|---------------|
| <div> <div>+</div> <div>SQL</div> </div> |          |               |
|                                          | count    | bigint        |
| 1                                        | 3        |               |

Die teuerste Unterkunft:

|    |                                                 |
|----|-------------------------------------------------|
| 59 | -- Aggregat                                     |
| 60 | SELECT AVG (Teilnehmerzahl) FROM Reise;         |
| 61 | SELECT COUNT (*) FROM Reise;                    |
| 62 | SELECT MAX (Unterkunftskosten) FROM Unterkunft; |
| 63 | SELECT MIN (Transportkosten) FROM Transport;    |
| 64 | SELECT SUM (Etappenkosten) FROM Etappe;         |

| Data Output                              | Messages | Notifications    |
|------------------------------------------|----------|------------------|
| <div> <div>+</div> <div>SQL</div> </div> |          |                  |
|                                          | max      | double precision |
| 1                                        | 830      |                  |

Der günstigste Transport:

|    |                                                 |
|----|-------------------------------------------------|
| 59 | -- Aggregat                                     |
| 60 | SELECT AVG (Teilnehmerzahl) FROM Reise;         |
| 61 | SELECT COUNT (*) FROM Reise;                    |
| 62 | SELECT MAX (Unterkunftskosten) FROM Unterkunft; |
| 63 | SELECT MIN (Transportkosten) FROM Transport;    |
| 64 | SELECT SUM (Etappenkosten) FROM Etappe;         |

| Data Output                              | Messages | Notifications    |
|------------------------------------------|----------|------------------|
| <div> <div>+</div> <div>SQL</div> </div> |          |                  |
|                                          | min      | double precision |
| 1                                        | 23       |                  |

Die Summe aller 'Etappenkosten':

```
59 -- Aggregat
60 SELECT AVG (Teilnehmerzahl) FROM Reise;
61 SELECT COUNT (*) FROM Reise;
62 SELECT MAX (Unterkunftskosten) FROM Unterkunft;
63 SELECT MIN (Transportkosten) FROM Transport;
64 SELECT SUM (Etappenkosten) FROM Etappe;
```

Data Output Messages Notifications

SQL

|   | sum<br>double precision |
|---|-------------------------|
| 1 | 2736.18                 |

## Group-by-Klausel

Bei dem Group-By Testszenario, kann man alle vorhandenen Tupel gruppieren. Dies heißt, es werden von einer Spalte alle Tupels angezeigt, ohne Duplikate. Hier ein Testszenario zu 'Sachgebiet':

```
378 -- Group By Beispiel, Sachgebiet mit Mitarbeiteranzahl
379 SELECT Sachgebiet, COUNT(Person_Id) AS Anzahl_Mitarbeiter
380 FROM Mitarbeiter
381 GROUP BY Sachgebiet;
```

Data Output Messages Notifications

SQL

|   | sachgebiet<br>character varying (30) | anzahl_mitarbeiter<br>bigint |
|---|--------------------------------------|------------------------------|
| 1 | Nordamerikareisen                    | 1                            |
| 2 | Unbestimmt                           | 1                            |
| 3 | Afrikareisen                         | 1                            |
| 4 | Europareise                          | 1                            |
| 5 | Asienreisen                          | 1                            |

## Geschachteltes Select-Statement

The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, query execution, and settings. The 'Query' tab is active, displaying the following SQL code:

```
383 -- geschachteltes Select-Statement, Reise mit höchstem Gesamtpreis
384 SELECT Reise_Id, Teilnehmerzahl, Gesamtpreis,
385        (SELECT Name FROM Person WHERE Person.Person_Id = Reise.Person_Id) AS Kunde
386 FROM Reise
387 WHERE Gesamtpreis = (SELECT MAX(Gesamtpreis) FROM Reise);
```

Below the query editor, the 'Data Output' tab is active, showing a table with the results of the query:

|   | reise_id<br>[PK] integer | teilnehmerzahl<br>integer | gesamtpreis<br>double precision | kunde<br>character varying (30) |
|---|--------------------------|---------------------------|---------------------------------|---------------------------------|
| 1 | 444                      | 3                         | 1530.79                         | Mueller                         |

Mit dieser Abfrage wird die Reise mit dem höchsten Gesamtpreis ermittelt und angezeigt. Im Hauptselect werden die Reise\_Id, die Teilnehmerzahl und der Gesamtpreis aus der Tabelle 'Reise' abgerufen. Eine Subquery wird verwendet, um den Namen des Kunden zu ermitteln, der die Reise gebucht hat. Die zweite Subquery ermittelt den höchsten Gesamtpreis aller Reisen. In der WHERE-Klausel wird dann die Reise ausgewählt, deren Gesamtpreis dem maximalen Gesamtpreis entspricht.

## Erfolgreiche Transaktion

Bei unserem Beispiel einer erfolgreichen Transaktion, ändern wir den Versicherungsstatus einer Reise von 'true' zu 'false'. Die Reise hat nun keine Versicherung mehr. Hier die Tabelle vor der Transaktion:

The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, query execution, and settings. The 'Query' tab is active, displaying the following SQL code:

```
390 -- Erfolgreiche Transaktion
391 -- 444 hatte vorher eine Versicherung gebucht: hier wird es geändert
392 BEGIN Transaction;
393 UPDATE Reise SET Versicherung = FALSE WHERE Reise_Id = 444;
394 COMMIT;
395 SELECT * FROM Reise;
```

Below the query editor, the 'Data Output' tab is active, showing a table with the results of the query:

|   | reise_id<br>[PK] integer | teilnehmerzahl<br>integer | buchungsdatum<br>timestamp without time zone | versicherung<br>boolean | gesamtpreis<br>double precision | person_id<br>character varying (30) |
|---|--------------------------|---------------------------|----------------------------------------------|-------------------------|---------------------------------|-------------------------------------|
| 1 | 444                      | 3                         | 2025-02-12 20:46:23.489155                   | true                    | 1530.79                         | K001                                |
| 2 | 446                      | 2                         | 2025-02-12 20:46:23.489155                   | true                    | 745.89                          | K005                                |
| 3 | 447                      | 2                         | 2025-02-12 20:46:23.489155                   | true                    | 459.5                           | K007                                |

Dann führen wir die Transaktion aus:

```

390 -- Erfolgreiche Transaktion
391 -- 444 hatte vorher eine Versicherung gebucht: hier wird es geändert
392 BEGIN Transaction;
393 UPDATE Reise SET Versicherung = FALSE WHERE Reise_Id = 444;
394 COMMIT;
395 SELECT * FROM Reise;

```

Data Output Messages Notifications

COMMIT

Query returned successfully in 95 msec.

Und dann nochmal die Tabelle nach der Transaktion:

```

390 -- Erfolgreiche Transaktion
391 -- 444 hatte vorher eine Versicherung gebucht: hier wird es geändert
392 BEGIN Transaction;
393 UPDATE Reise SET Versicherung = FALSE WHERE Reise_Id = 444;
394 COMMIT;
395 SELECT * FROM Reise;

```

Data Output Messages Notifications

|   | reise_id<br>[PK] integer | teilnehmerzahl<br>integer | buchungsdatum<br>timestamp without time zone | versicherung<br>boolean | gesamtpreis<br>double precision | person_id<br>character varying (30) |
|---|--------------------------|---------------------------|----------------------------------------------|-------------------------|---------------------------------|-------------------------------------|
| 1 | 446                      | 2                         | 2025-02-12 20:46:23.489155                   | true                    | 745.89                          | K005                                |
| 2 | 447                      | 2                         | 2025-02-12 20:46:23.489155                   | true                    | 459.5                           | K007                                |
| 3 | 444                      | 3                         | 2025-02-12 20:46:23.489155                   | false                   | 1530.79                         | K001                                |

## Abgebrochene Transaktion

In diesem Beispiel wird eine Transaktion mit einem fehlerhaften INSERT-Befehl durchgeführt. Dabei wurden die Werte für Hausnummer und Postleitzahl vertauscht, sodass die Postleitzahl fälschlicherweise als Hausnummer und umgekehrt gespeichert werden würde.

Zunächst wird die Transaktion mit `BEGIN Transaction` gestartet, bevor der fehlerhafte Datensatz über den INSERT zur Adresse eingefügt wird. Mithilfe des SELECTS Statement lässt sich die Adress-Tabelle anzeigen:



SQL X Statistics X Dependencies X Dependents X Processes X Doku/postgres@P... X Doku2/postgres@PostgreSQL 17\* X

Doku2/postgres@PostgreSQL 17

Query Query History Scratch Pad

```

400 INSERT INTO Adresse (Adress_Id, StraSe, Hausnummer, PLZ, Stadt)
401 VALUES (9, 'Hauptstraße', 12345, '10', 'Berlin');
402 SELECT * FROM Adresse;
403 ROLLBACK;
404 SELECT * FROM Adresse;

```

Data Output Messages Notifications

|    | adress_id<br>[PK] integer | straße<br>character varying (30) | hausnummer<br>character varying (30) | plz<br>integer | stadt<br>character varying (30) |
|----|---------------------------|----------------------------------|--------------------------------------|----------------|---------------------------------|
| 1  | 12                        | Kirchstrasse                     | 55                                   | 2668           | Flensburg                       |
| 2  | 13                        | Landstrasse                      | 2                                    | 5832           | Heidelberg                      |
| 3  | 1                         | Knochenstraße                    | 3                                    | 24111          | Bremen                          |
| 4  | 2                         | Sandstraße                       | 29                                   | 29141          | Bremen                          |
| 5  | 3                         | Palaststraße                     | 16                                   | 20221          | Bremen                          |
| 6  | 4                         | Schulstraße                      | 8                                    | 22222          | Hamburg                         |
| 7  | 5                         | Taunusstraße                     | 7                                    | 41321          | Muenchen                        |
| 8  | 6                         | Spandauerstraße                  | 5                                    | 33123          | Berlin                          |
| 9  | 7                         | Baumstraße                       | 106                                  | 52675          | Essen                           |
| 10 | 9                         | Hauptstraße                      | 12345                                | 10             | Berlin                          |

Total rows: 10 of 10 Query complete 00:00:00.319 Ln 403, Col 10

Der "fehlerhafte" Eintrag wurde erst Mal in die Tabelle hinzugefügt.

Da die Werte jedoch falsch zugeordnet wurden, erfolgt ein ROLLBACK, wodurch die gesamte Transaktion zurückgesetzt wird. Eine erneute SELECT Abfrage zeigt, dass der fehlerhafte Eintrag nicht mehr existiert:

SQL X Statistics X Dependencies X Dependents X Processes X Doku/postgres@P... X Doku2/postgres@PostgreSQL 17\* X

Doku2/postgres@PostgreSQL 17

Query Query History Scratch Pad

```

397 -- Abgebrochene Transaktion
398 -- Fehlerhafte Aktion: Vertauschte Werte für Hausnummer und PLZ
399 BEGIN Transaction;
400 INSERT INTO Adresse (Adress_Id, StraSe, Hausnummer, PLZ, Stadt)
401 VALUES (9, 'Hauptstraße', 12345, '10', 'Berlin');

```

Data Output Messages Notifications

|   | adress_id<br>[PK] integer | straße<br>character varying (30) | hausnummer<br>character varying (30) | plz<br>integer | stadt<br>character varying (30) |
|---|---------------------------|----------------------------------|--------------------------------------|----------------|---------------------------------|
| 1 | 12                        | Kirchstrasse                     | 55                                   | 2668           | Flensburg                       |
| 2 | 13                        | Landstrasse                      | 2                                    | 5832           | Heidelberg                      |
| 3 | 1                         | Knochenstraße                    | 3                                    | 24111          | Bremen                          |
| 4 | 2                         | Sandstraße                       | 29                                   | 29141          | Bremen                          |
| 5 | 3                         | Palaststraße                     | 16                                   | 20221          | Bremen                          |
| 6 | 4                         | Schulstraße                      | 8                                    | 22222          | Hamburg                         |
| 7 | 5                         | Taunusstraße                     | 7                                    | 41321          | Muenchen                        |
| 8 | 6                         | Spandauerstraße                  | 5                                    | 33123          | Berlin                          |
| 9 | 7                         | Baumstraße                       | 106                                  | 52675          | Essen                           |

Total rows: 9 of 9 Query complete 00:00:00.292 Ln 399, Col 19

Durch den Einsatz von Transaktionen und dem gezielten Zurücksetzen fehlerhafter Änderungen wird sichergestellt, dass inkonsistente Daten nicht in der Datenbank gespeichert bleiben. Dieses Beispiel verdeutlicht die Bedeutung von Transaktionen in relationalen Datenbanken.

# Beantwortung der theoretischen Fragen

## 1. Was versteht man im Datenbankkontext unter einer Transaktion?

Eine Transaktion ist eine logische Einheit aus einer oder mehrerer Datenbankoperationen, die als unteilbare Aktion betrachtet wird. Sie sorgt für Konsistenz und Integrität der Datenbank gemäß dem ACID-Prinzip und kann entweder vollständig abgeschlossen (Commit) oder bei Fehlern zurückgesetzt (Rollback) werden.

## 2. Erläutern Sie das ACID-Paradigma.

Das ACID-Paradigma beschreibt vier essenzielle Eigenschaften einer Transaktion in einem Datenbanksystem:

- Atomicity (Atomarität): Die Transaktion wird entweder vollständig ausgeführt oder gar nicht.
- Consistency (Konsistenz): Nach einer abgeschlossenen Transaktion befindet sich die Datenbank wieder in einem konsistenten Zustand.
- Isolation: Parallel ausgeführte Transaktionen beeinflussen sich nicht gegenseitig.
- Durability (Dauerhaftigkeit): Nach einem erfolgreichen Commit bleiben die Änderungen dauerhaft in der Datenbank gespeichert, selbst bei einem Systemabsturz.

## 3. Welche (drei) Concurrency-Probleme können bei einem verteilten Datenbankbetrieb auftreten? Erläutern Sie diese (inkl. ihres zeitlichen Verlaufs) jeweils anhand eines selbstgewählten Beispiels.

**Verlorengegangene Änderung:** Bei dieser Concurrency-Problem ändern zwei Transaktionen dieselbe Datenbank-Zeile fast gleichzeitig. Dadurch überschreibt die zweite Transaktion die erste, sodass sie verloren geht.

Beispiel:

Zwei Mitarbeiter wollen die Adresse eines Kunden ändern. Dabei will Mitarbeiter 1 die Straße und Mitarbeiter 2 die Hausnummer ändern. Wenn diese Transaktionen gleichzeitig geschehen sind, dann kann die Veränderung des Mitarbeiter 1 verloren gehen, wenn Mitarbeiter 2 seine Änderung speichert.

Zeitlicher Verlauf:

| Zeitpunkt | Transaktion TA1                                          | Transaktion TA2                                          | Datenbestand (Adresse)                                           |
|-----------|----------------------------------------------------------|----------------------------------------------------------|------------------------------------------------------------------|
| t1        | Liest Adresse (Straße: "Kirchstrasse", Hausnummer: "55") | Liest Adresse (Straße: "Kirchstrasse", Hausnummer: "55") | Straße: "Kirchstrasse", Hausnummer: "55"                         |
| t2        | Ändert Straße zu "Hauptstraße"                           |                                                          | Straße: "Hauptstraße", Hausnummer: "55" (nur im Puffer von TA1)  |
| t3        |                                                          | Ändert Hausnummer zu "10"                                | Straße: "Kirchstrasse", Hausnummer: "10" (nur im Puffer von TA2) |
| t4        | Speichert Änderung                                       |                                                          | Straße: "Hauptstraße", Hausnummer: "55"                          |
| t5        |                                                          | Speichert Änderung                                       | Straße: "Kirchstrasse", Hausnummer: "10"                         |
| Ergebnis  | Änderung von TA1 (Straße) geht verloren!                 |                                                          |                                                                  |

**Abhängigkeit von nicht abgeschlossenen Transaktionen:** Dieses Problem entsteht, wenn eine Transaktion liest Daten, die von einer anderen, noch nicht abgeschlossenen Transaktion geändert wurden. Wenn die erste Transaktion zurückgesetzt wird, sind die gelesenen Daten ungültig.

Beispiel:

Wenn ein Mitarbeiter eine Adresse eines Kunden einlesen möchte, während ein anderer Mitarbeiter diese ändert. Wenn diese Veränderung jetzt noch zurückgesetzt wird, hat der erste Mitarbeiter ungültige Daten eingelesen.

Zeitlicher Verlauf:

| Zeitpunkt | Transaktion TA1                                          | Transaktion TA2                                         | Datenbestand (Adresse)                                          |
|-----------|----------------------------------------------------------|---------------------------------------------------------|-----------------------------------------------------------------|
| t1        | Ändert Adresse (Straße: "Hauptstraße", Hausnummer: "10") |                                                         | Straße: "Hauptstraße", Hausnummer: "10" (nur im Puffer von TA1) |
| t2        |                                                          | Liest Adresse (Straße: "Hauptstraße", Hausnummer: "10") | Straße: "Hauptstraße", Hausnummer: "10"                         |
| t3        | Rollback (Änderung wird rückgängig gemacht)              |                                                         | Straße: "Kirchstrasse", Hausnummer: "55"                        |
| Ergebnis  | TA2 hat ungültige Daten gelesen!                         |                                                         |                                                                 |

**Inkonsistenz der Daten:** Dieses Problem entsteht, wenn eine Transaktion Daten liest, während eine andere Transaktion diese ändert. Dies führt dann zu inkonsistenten und fehlerhaften Ergebnissen.

Beispiel:

Wenn ein Mitarbeiter den 'Gesamtpreis' einer Reise liest, aber währenddessen ein andere Mitarbeiter die 'Etappenkosten' ändert. Das führt zu einer inkonsistenten 'Gesamtpreis'.

Zeitlicher Verlauf:

| Zeitpunkt | Transaktion TA1                                   | Transaktion TA2                           | Datenbestand (Adresse)                                    |
|-----------|---------------------------------------------------|-------------------------------------------|-----------------------------------------------------------|
| t1        | Liest Etappenkosten (Etappe 301: 250.00)          |                                           | Etappe 301: 250.00                                        |
| t2        |                                                   | Ändert Etappenkosten (Etappe 301: 300.00) | Etappe 301: 300.00 (nur im Puffer von TA2)                |
| t3        | Liest Etappenkosten (Etappe 302: 280.00)          |                                           | Etappe 302: 280.00                                        |
| t4        | Berechnet Gesamtkosten (250.00 + 280.00 = 530.00) |                                           | Gesamtkosten: 530.00 (basierend auf inkonsistenten Daten) |
| Ergebnis  | TA1 hat inkonsistente Daten gelesen!              |                                           |                                                           |

#### 4. Was versteht man unter einem S-Lock, einem X-Lock und einem Deadlock?

S-Lock, auch als Shared Lock genannt, ist eine Sperre, die es erlaubt, mit mehreren Transaktionen gleichzeitig auf dieselbe Ressource zuzugreifen. Dies ist aber nur mit Leseoperationen wie 'SELECT' möglich.

X-Lock, das Exclusive Lock heißt, ist eine Sperre, die es nur einer Transaktion erlaubt, exklusive auf eine Ressource zuzugreifen, welche sowohl für den Leseoperationen als auch für die Schreiboperationen geht.

Deadlock tritt dann auf, wenn mindestens zwei Transaktionen gegenseitig auf die Freigabe von Sperrern warten, die die jeweils andere Transaktion hält. Dadurch kommen beide Transaktionen zum Stillstand und können beide nicht mehr fortgesetzt werden.

## 5. Was versteht man im Datenbankkontext unter dem Begriff „Zugriffslücke“?

Der Begriff „Zugriffslücke“ beschreibt den erheblichen Unterschied in der Geschwindigkeit, mit der verschiedene Speicherarten Daten abrufen können. Zum Beispiel dauert der Zugriff auf den Hauptspeicher nur etwa 100 Nanosekunden, während eine Festplatte für denselben Vorgang etwa 10 Millionen Nanosekunden benötigt, welches 100.000-mal länger dauert. Diese enorme Zeitdifferenz zwischen zwei Speicherstufen wird als Zugriffslücke bezeichnet.